ENGINEERING PORTFOLIO

Ariadne Colina-Valeri

Hi! My name is Ariadne, and I'm a Junior studying Electrical Engineering at Tufts University. I have experience in signal processing, embedded systems and machine learning, from restoring RF signals with a Wiener filter to building a real-time audio effects board and an autonomous robot. I'm looking for internships in those fields.

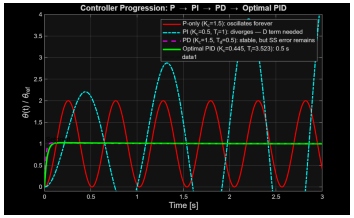


ariadnecolina@gmail.com
(813) 847-5642

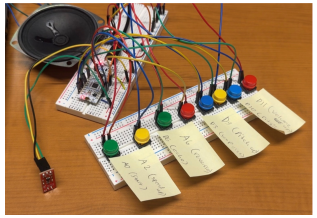
SELECTED PROJECTS



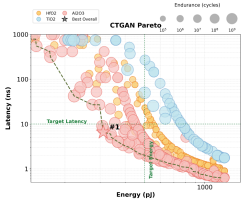
Autonomous Robot
page 2



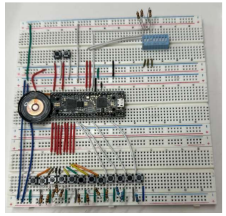
UAV Pitch Control
page 3



DJ Board on an STM32
page 4



CTGAN for RRAM
page 5



FPGA Synthesizer
page 6

PROJECT 1 | IN PROGRESS | JUNIOR DESIGN | SEPT. 2026 TO PRESENT

Autonomous ESP32 Robot

THE CHALLENGE

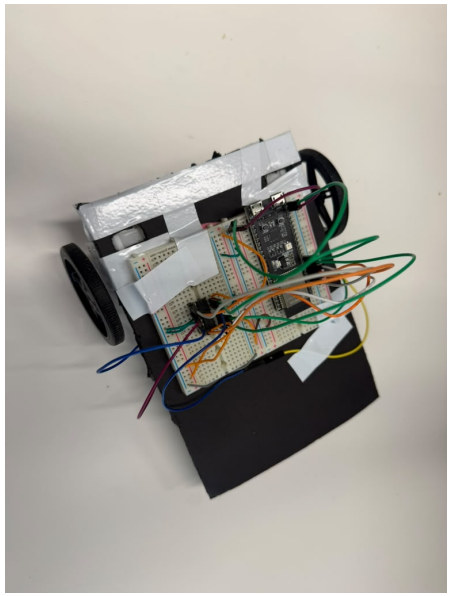
The robot has to follow a colored lane and avoid obstacles with no one controlling it, and accept commands only from our own team.

WHAT I'M DOING

I built a 7-state state machine on the ESP32 that authenticates every websocket command with our team key. For obstacles, an IR LED and photodetector measure reflected light, and I subtract the room's background light from each reading. Now I'm building the lane color sensing. Colored LEDs light the track, and the robot compares their reflections to tell black, yellow, red and blue apart.

NEXT

Assembling the robot inside the ladybug body I'm designing in SolidWorks, on a custom ESP32 PCB, then programming lane following. For the go-beyond part, sound will control its movements, with volume and certain notes each doing something different.



The robot so far, wired on a breadboard around the ESP32



The ladybug body the robot will be assembled in

UAV Pitch Control and RF Signal Restoration

THE CHALLENGE

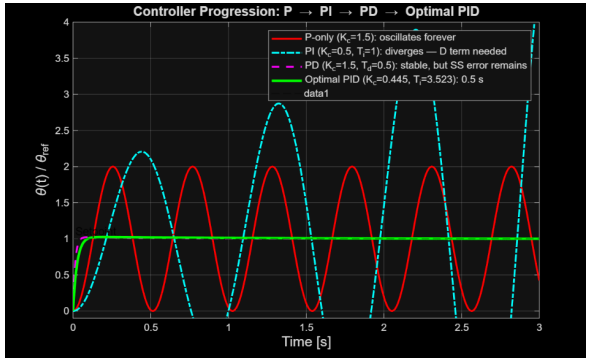
The UAV has to stay level while it collects RF signals, and the signals blurred by its shaking have to be restored.

WHAT I DID

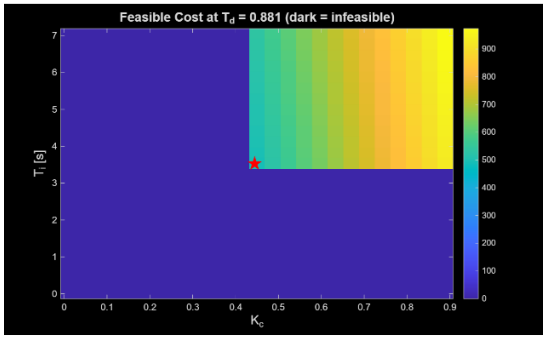
Routh-Hurwitz analysis proved that P and PI controllers can never keep the UAV stable, so the design needed a full PID controller. To find the best PID settings, I tested 15,625 combinations of gains. With the controller in place, I restored a blurred RF signal using a Wiener filter in MATLAB. Connecting the two parts showed that the longer the controller takes to settle, the longer the signal stays blurred.

THE RESULT

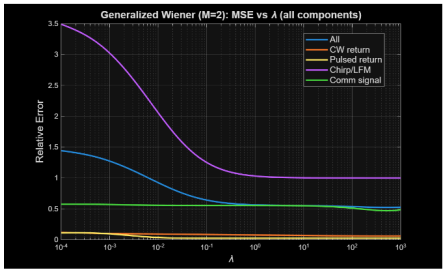
The PID controller rises in 52ms with only 2.6% overshoot and an 88.3° phase margin. The Wiener filter cut the error by 77% compared with a direct inverse, and a controller that settles in 2s instead of 0.5s makes the recovery 1.5× worse.

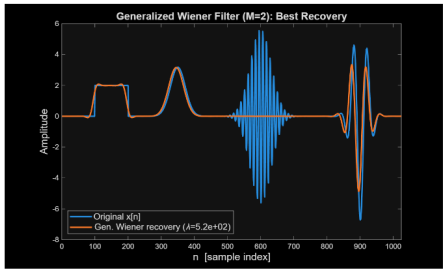


P, PI, PD and PID controllers



Cost of each gain pair; the star is the optimum





Wiener filter error (left) and the restored RF signal over the original (right)

DJ Board on an STM32

(Audio Recorder and Effects Processor)

THE CHALLENGE

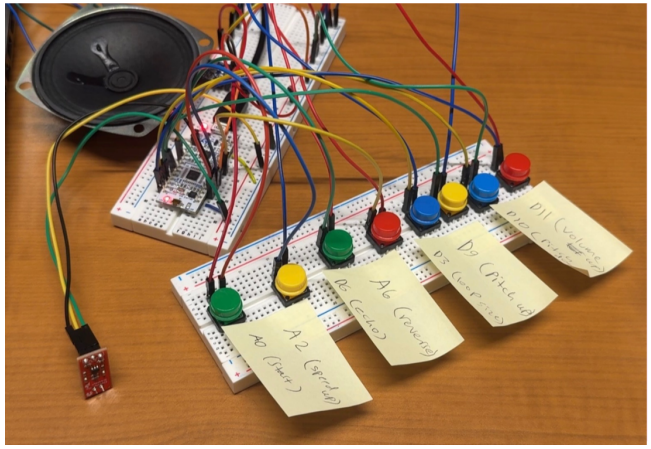
The board has to record sound and play it back in real time with 7 effects (echo, reverse, speed, loop size, pitch up, pitch down, volume), all within the STM32's 64KB of on-chip memory.

WHAT I DID

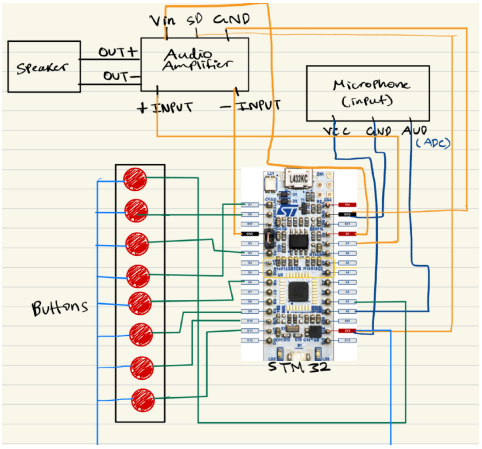
Recording at 8kHz left room for only 2 seconds of audio, so I lowered the sample rate to 4kHz to store 5 seconds. Since every effect depends on a clean recording, the main problem to solve was that playback came out with a high-pitched hiss and clicks. To fix the hiss, I wrote a state machine driven by a 4kHz timer interrupt so every sample is recorded and played at the same rate, and lengthened the ADC sampling time to 12.5 cycles so each microphone reading has time to settle. The clicks came from sudden jumps in the speaker signal, so I held the speaker at mid-rail, its resting level, between clips and faded each clip in and out over 256 samples (64ms).

THE RESULT

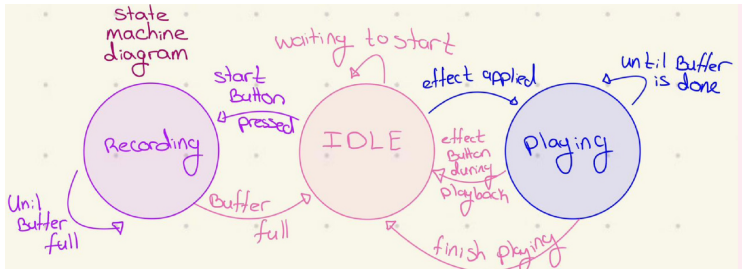
The board records 5 seconds of 12-bit audio in a 40KB buffer, plays all 7 effects, and its playback matched the original recording on the oscilloscope.



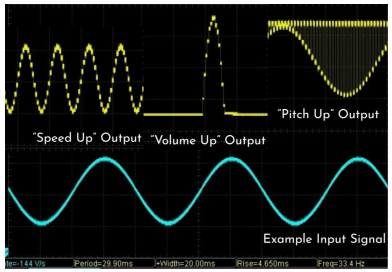
The board



Hardware: wiring diagram



Software: the state machine



Oscilloscope captures of the effects

STM32: every sample moves on a 4kHz timer



CTGAN Design Discovery for RRAM

THE CHALLENGE

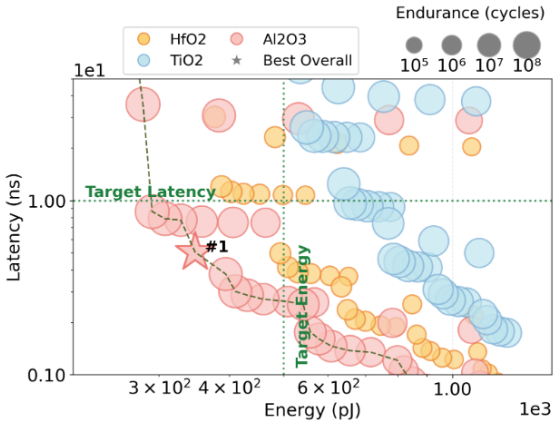
The project tests whether a more aggressive AI model can find better RRAM memory designs than a published framework, without inventing designs that cannot physically exist.

WHAT I DID

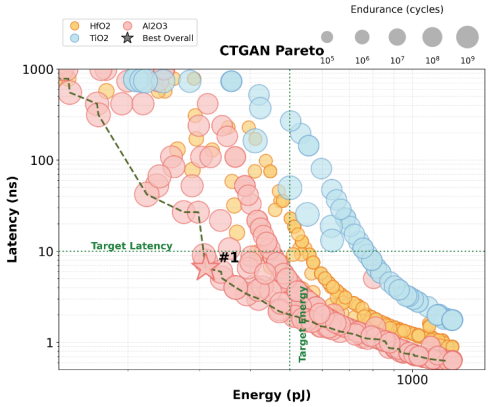
The published framework used a CVAE, an AI model that averages its training data. Averaging hides the unusual designs, and those are where the best performance is, so I replaced it with a CTGAN, a model that searches those unusual designs more aggressively. To keep the CTGAN from inventing designs that cannot physically exist, I used the framework's physics model (PINN) to reject them. The CTGAN failed with 500 training samples and was still noisy with 2,500, so I grew the data to 5,000, where it worked for all 3 materials (Al_2O_3, HfO_2 and TiO_2).

THE RESULT

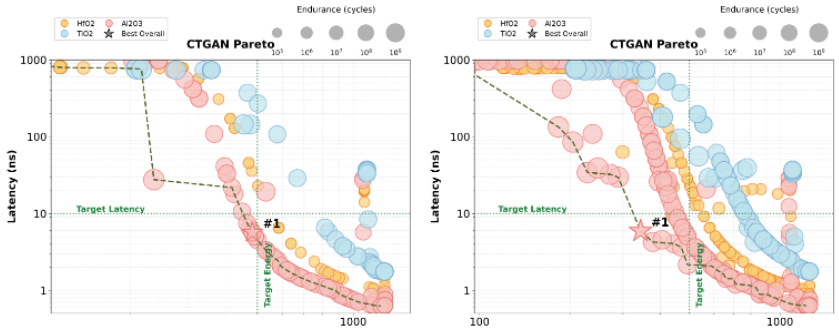
My best Al_2O_3 design lasted 25% longer than the published one (4.84 billion cycles against 3.85 billion) and used 11% less energy, though it switched a little slower (6.57ns against 4.07ns).



Published CVAE results



My CTGAN with 5,000 samples



The CTGAN with 500 samples (left) and 2,500 (right)

FPGA Audio Synthesizer

THE CHALLENGE

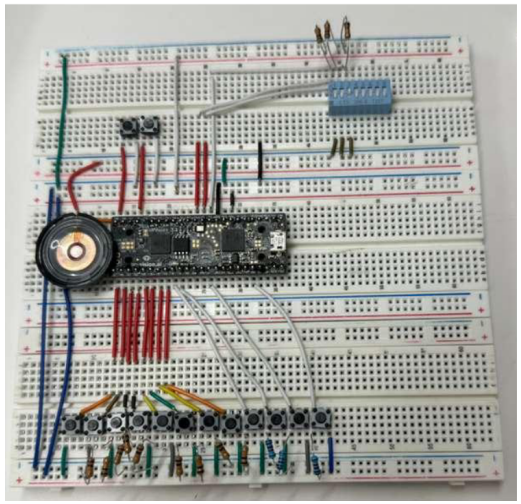
The synthesizer has to play notes and chords from a MIDI keyboard across 8 octaves on an FPGA.

WHAT I DID

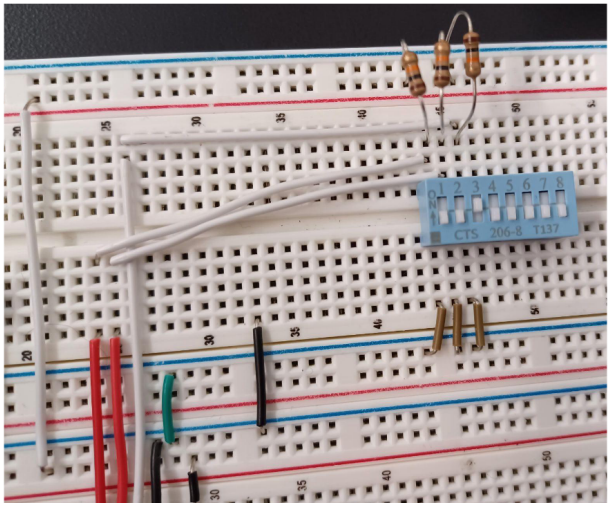
Calculating every note by division required over 250% of the FPGA's resources. Since each octave doubles the frequency, and dividing by 2 is equivalent to a bit shift, I calculated one octave by hand and derived the remaining seven with bit shifts, which brought the design within the chip's capacity. Testing with a tuner showed every note an octave too low, meaning the clock ran at twice the rate the code assumed, so I rebuilt the scale for the real clock. Widening the counter that holds each note's period extended the range below 92Hz down to 16Hz. Chords of more than 2 notes, or in low octaves, turned into static; plotting them in Desmos showed the output was not following the summed wave, so I mixed the notes with PWM on a 3-bit bus, which gave the best balance of volume and clarity.

THE RESULT

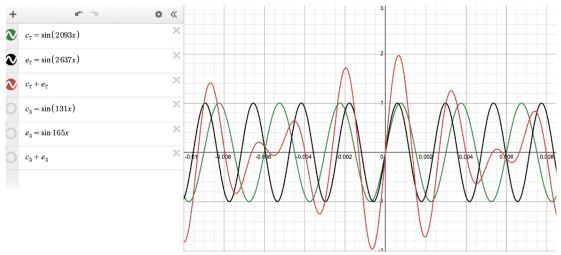
The keyboard plays all 12 notes across 8 octaves (16Hz to 8kHz), chords of 5 to 6 notes in the lowest octave, and every note at once in the highest.



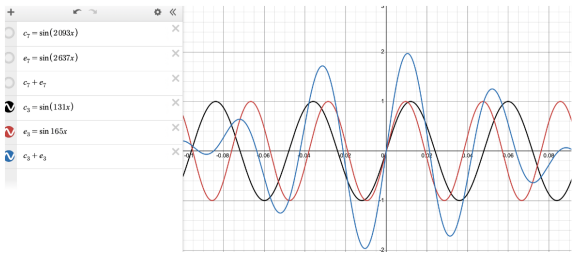
The complete board



The switch that selects the octave



C + E summed in octave 7 (red)



C + E summed in octave 3 (blue)